

C Programming

1_conditions

Keyboard input

scanf() - ф-ия ввода с клавиатуры, состоит из двух слов `scan` и `f` (format).

Для записи в переменную значения введенного с клавиатуры в scanf перед переменной необходимо ставить знак **амперсанда** '&', где & в языке Си это **операция взятия адреса переменной**

Keyboard input

```
int a;  
printf("type a:");  
scanf("%d", &a);  
printf("a = %d\n", a);
```

Keyboard input

```
int x, y;  
printf("Give me 2 numbers: ");  
scanf("%d%d", &x, &y);
```

Comments

В программировании есть возможность комментировать код - такой код **компилятор/интерпретатор полностью игнорирует**, т.е. комментарии используются разработчиками для разработчиков

// Вся эта строка будет игнорироваться компилятором

/* Такой комментарий открывается через слеш звездочку
и будет комментировать все
пока не встретим звездочку слеш */

Type casting (Приведение типов)



implicit (неявное)

```
int x = 5;  
double y = 12.3;  
y = x;  
x = y;
```

```
double z = x / 2; // z = 2.0
```

explicit (явное)

```
int a = 5;  
double b = 12.3;  
b = (double) a;  
a = (int) b;
```

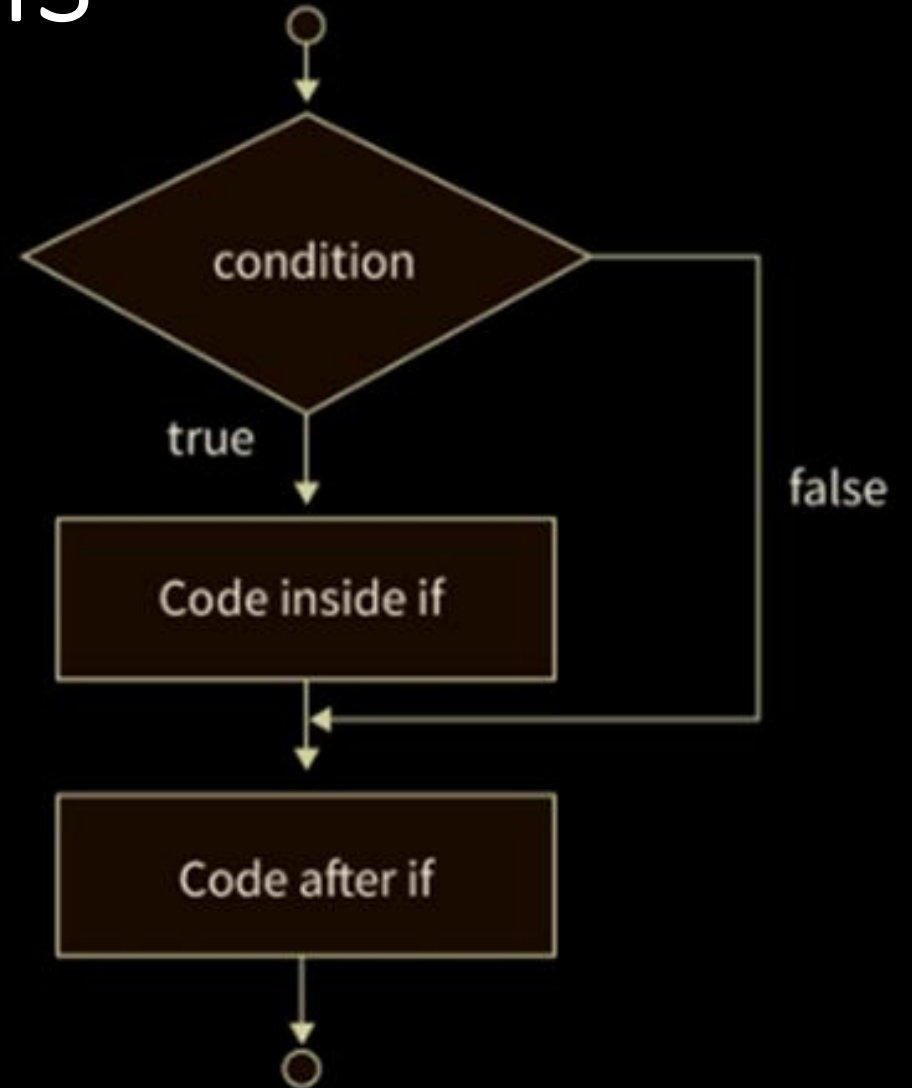
```
double c = (double)a / 2; // c = 2.5
```

Conditions

В программировании **условия** позволяют контролировать ход выполнения программы на основе этих условий

Чаще всего можно встретить следующие типы условий

- if, if-else и прочие
- switch-case
- ternary operator (тернарный оператор)



If statement

```
if (condition) {  
    // code if condition is true  
}  
// code after
```

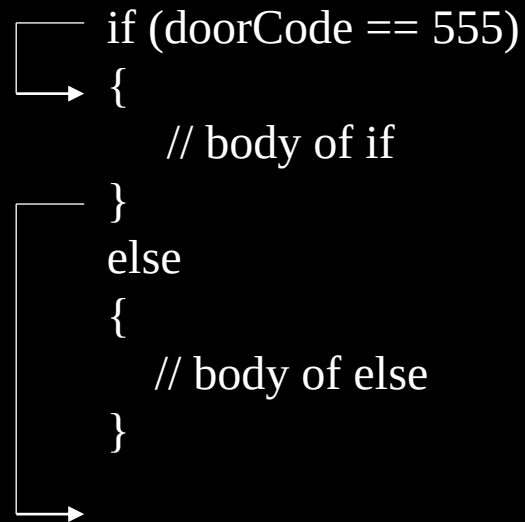
```
if (a > 5) {  
    printf("a > 5!\n");  
}  
// code after
```

If-else

Expression is true (истина)

```
int doorCode = 555;
```

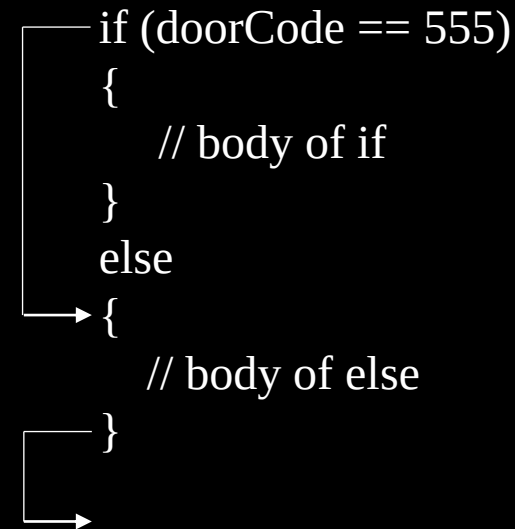
```
if (doorCode == 555)
{
    // body of if
}
else
{
    // body of else
}
```



Expression is false (ложь)

```
int doorCode = 123;
```

```
if (doorCode == 555)
{
    // body of if
}
else
{
    // body of else
}
```



If-else-if

```
if (c == 'y') {  
    // code for condition1 is true (c = 'y')  
} else if (c == 'n') {  
    // code for condition2 (c != 'y' and c = 'n')  
} else {  
    // code for else (c != 'y' and c != 'n')  
}  
// code after
```

True and False (1 and 0)

```
if (1) {  
    // ?  
} else {  
    // ?  
}
```

```
if (0) {  
    // ?  
} else {  
    // ?  
}
```

```
if (-1) {  
    // ?  
} else {  
    // ?  
}
```

1 == истина, 0 == ложь, -1 = ?

Logical Operators

&&	И (AND)
	ИЛИ (OR)
!	НЕ (NOT)
==, !=	Равно (EQUAL), Не равно (NOT EQUAL)
>, <	Больше, Меньше
>=, <=	Больше либо равно, Меньше либо равно

Logical Operators

Операторы `&&` (И) и `||` (ИЛИ) позволяют проверять несколько условий внутри одного блока. Вариант слева заменяем на вариант справа.

```
if (a == 'y') {  
  
} else if (a == 'Y') {  
  
}
```

```
if (a == 'y' || a == 'Y') {  
  
}
```

Важно: ставить ровно две палки `||` или ровно два амперсанда `&&`, т.к. в Си одна палка `|` и один амперсанд `&` - это не **логические операторы**

Precedence	Operator	Description
1	::	Scope resolution
2	a++ a-- type() type{} a() a[] . ->	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access
3	++a --a +a -a ! ~ (type) *a &a sizeof co_await new new[] delete delete[]	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of await-expression (C++20) Dynamic memory allocation Dynamic memory deallocation
4	.* ->*	Pointer-to-member
5	a*b a/b a%b	Multiplication, division, and remainder
6	a+b a-b	Addition and subtraction
7	<< >>	Bitwise left shift and right shift
8	<=>	Three-way comparison operator (C++20)
9	< <= > >=	For relational operators < and ≤ and > and ≥ respectively
10	== !=	For equality operators = and ≠ respectively
11	&	Bitwise AND
12	^	Bitwise XOR (exclusive or)
13		Bitwise OR (inclusive or)
14	&&	Logical AND
15		Logical OR
16	a?b:c throw co_yield = += -= *= /= %= <<= >>= &= ^= =	Ternary conditional throw operator yield-expression (C++20) Direct assignment (provided by default for C++ classes) Compound assignment by sum and difference Compound assignment by product, quotient, and remainder Compound assignment by bitwise left shift and right shift Compound assignment by bitwise AND, XOR, and OR
17	,	Comma

Если не знаете (или не уверены) в каком порядке будут выполнены действия - используйте `()` для обозначения задуманного вами порядка действий. **Лишняя пара скобок хуже не сделает**

is even?

```
int number = 5;

if (number % 2 == 0) {
    printf("%d is even.\n", number);
} else {
    printf("%d is odd.\n", number);
}
```

Символ % - арифметический оператор, дающий остаток от деления целых чисел
(1 % 2 = 1, 2 % 2 = 0, 3 % 2 = 1, 4 % 2 = 0 и т.д..)

Так не надо

```
private bool IsEven(int number){  
    if (number == 1) return false;  
    else if (number == 2) return true;  
    else if (number == 3) return false;  
    else if (number == 4) return true;  
    else if (number == 5) return false;  
    else if (number == 6) return true;  
    else if (number == 7) return false;  
    else if (number == 8) return true;  
    else if (number == 9) return false;  
    else if (number == 10) return true;  
    else if (number == 11) return false;  
    else if (number == 12) return true;  
    else if (number == 13) return false;  
    else if (number == 14) return true;  
    else if (number == 15) return false;  
    else if (number == 16) return true;  
    else if (number == 17) return false;
```

Logical error (=)

```
// bad
if (a = 6) {
    // Условие всегда истинно
}
```

```
// not bad
if (6 == a) {
    // 6 == a оберегает от ошибок
}
```

Logical error (!= || !=)

Для любых var всегда истина

```
if (var != s1 || var != s2) {  
    // ...  
} else {  
    // ...  
}
```

var	var != s1	var != s2	
s1	0	1	1
s2	1	0	1
none	1	1	1

Logical error (== && ==)

Для любых var всегда ложь

```
if (var == s1 && var == s2) {  
    // ...  
} else {  
    // ...  
}
```

var	var == s1	var == s2	&&
s1	1	0	0
s2	0	1	0
none	0	0	0

Logical error (== || !=)

Результат не зависит от var == s1

```
if (var == s1 || var != s2) {  
    // ...  
} else {  
    // ...  
}
```

var	var == s1	var != s2	
s1	1	1	1
s2	0	0	0
none	0	1	1

Logical error (== && !=)

Результат не зависит от `var == s2`

```
if (var == s1 && var != s2) {  
    // ...  
} else {  
    // ...  
}
```

var	var == s1	var != s2	&&
s1	1	1	1
s2	0	0	0
none	0	1	0

{ } и if-else

Фигурные скобки в условных выражениях на языке Си опциональны и если их не поставить, то это будет означать, что мы хотим выполнить самую первую ближайшую инструкцию под if (или else)

```
if (a == 5)
    printf("a == 5!\n"); // Выполнится если a равно 5

    printf("hello!\n"); // Выполнится в любом случае (табы/пробелы не важны)
```

```
if (a == 5)
    printf("a == 5!\n"); // Ок
    printf("hello!\n"); // Ошибка! Больше одной инструкции без фигурных скобок
else
    printf("else here"); // Ок
```

Ternary Operator

Тернарный оператор (от латинского **тройной**). Означает следующее: Если **Q**, то **P1**, иначе **P2**

if .. else

```
int a = 3, b = 5;  
int max;
```

```
if (a > b) {  
    max = a;  
} else {  
    max = b;  
}
```

Ternary Operator

```
int a = 3, b = 5;  
int max;
```

```
max = a > b ? a : b;
```

switch .. case

```
if (a == 'a') {  
    // ...  
} else if (a == 'b') {  
    // ...  
} else if (a == 'c') {  
    // ...  
} else if (a == 'd') {  
    // ...  
} else {  
    // ...  
}
```

```
switch (a) {  
    case 'a':  
        // ...  
        break;  
    case 'b':  
        // ...  
        break;  
    case 'c':  
        // ...  
        break;  
    case 'd':  
        // ...  
        break;  
    default:  
        // ...  
}
```

- Если не поставить **break**, то после выполнения кода текущего case вы провалитесь в нижестоящий case;
- break (ломать) – остановить/выйти из switch;
- switch-case использует **хэш-таблицы** и **бинарный поиск** под капотом;

Branch prediction

Предсказатель переходов

```
graph TD; A[Предсказатель переходов] --> B[Статическое предсказание]; A --> C[Динамическое предсказание];
```

Статическое предсказание

Выбираем одну из веток и
только она в кэш кладется

Динамическое предсказание

Набираем статистику тех кто чаще
выполняется и их кладем в кэш

Branch misprediction and cache misses

```
if (e1) {  
    /*...*/  
    if (e2) {  
        /*...*/  
    } else if (e2_2) {  
        if (e3) { if (e4) { /*...*/ } else  
{ /*...*/ } }  
    } else if (e2_3) {  
        /*...*/  
    } else if (e2_4) /*...*/ else if (e2_5)  
/*...*/ else /*...*/  
    /*...*/  
} else {  
    /*...*/  
}  
/*...*/
```

Процессор такому не обрадуется (**особенно если такое условие проверяется много раз**) и будет частенько простаивать из-за промахов, поэтому обычно **хорошим тоном** является не писать много **вложенных условий** (условия в условиях), а ограничиваться до 3-4, если выходит больше, то возможно мы делаем что-то не правильно

Подобные промахи процессора в кэше называют **cache miss (промах кэша)**, когда процессор вынужден загрузить данные из более медленной памяти (например оперативной памяти), т.к. нужных данных не оказалось в кэше процессора